WhatsApp CRYPT14 mobile extraction update

Date: 6 October 2026

What is included

The existing mobile pipeline already had authenticated CRYPT12, CRYPT14 and CRYPT15 decoding and authorized key collection. This update preserves decoded databases for imported encrypted backups, records per-backup verification in intake status, shows outcomes in the UI, and adds an offline case-folder test and a synthetic demonstration.

A CRYPT14 file is already encrypted. The required operation is decryption. The file cannot reveal its AES key. The test exports a matching key only when that key was supplied separately or collected from the case folder. There is no universal key that decrypts every account or every kind of mobile data.

Mobile workflow

1. Acquire the mobile data and any accessible WhatsApp private app files through the existing authorized acquisition workflow. A public/shared-storage-only extraction normally will not include the private key.

2. Supply the matching key in intake, or allow the existing key discovery to collect recognized key files from the acquisition. Android WhatsApp commonly stores the CRYPT14 key at /data/data/com.whatsapp/files/key. This is app-private storage; ordinary file transfer does not expose it. This update does not bypass Android access controls.

3. Start mobile analysis, or use the existing reprocess action after adding a key. Each msgstore backup is tried against collected candidates independently. Matching keys are checked using authenticated decryption and a valid SQLite payload.

4. Inspect intake backup outcomes. A key with a valid format is not marked as a verified backup match until authentication succeeds. Missing keys, wrong keys, truncated or modified backups retain a blocked reason.

5. The parser preserves decoded msgstore.db in the job's derived evidence storage and parses supported chat records into the existing artifact pipeline. Source and decoded SHA-256 hashes record provenance. Storage failures are reported separately even when chat parsing succeeds.

The storage URI is retained in protected job metadata and the encrypted-backup artifact. This update does not add a new browser download button or a public key download endpoint. Original source files are unchanged.

Windows test commands

Open PowerShell at the extracted project root. Use a Python environment with the project's backend dependencies installed. Docker users can run the tests inside their existing backend container instead of installing the full GPU stack locally.

Run the 88 focused crypto/intake/delivery tests:

```
.\Test-Crypt14.ps1
```

Run the included synthetic demo (contains a public test key, not real phone data):

```
.\Test-Crypt14.ps1 -CaseFolder '.\backend\tests\fixtures\crypt14_demo' -OutputFolder
'.\crypt14-demo-output' -ExportKeyHex
```

Expected output: Backups=1 decrypted=1 blocked=0. The decrypted database SHA-256 is 4fe2e23f4f3e42f4ec03ccb4b743e0650038157088247bbce2a77fc945d6db4e. A parsed message reads 'independent crypt fixture chat'.

Run a real acquired case with a separately acquired key:

```
.\Test-Crypt14.ps1 -CaseFolder 'D:\Evidence\Case01' -OutputFolder 'D:\Results\Case01' -KeyFile
'D:\Keys\whatsapp-key' -ExportKeyHex
```

Use an output folder separate from original evidence. Omit -KeyFile when the matching key is already in a recognized path inside the case folder. Omit -ExportKeyHex to avoid creating an additional key file. Specify -Python with the path to your

backend virtual-environment python.exe if necessary. Test-Crypt14.cmd forwards the same arguments to PowerShell.

Linux equivalent:

```
python backend/scripts/test_crypt14_case.py --case-folder /evidence/case01 --output /results/case01
--key-file /keys/whatsapp-key --export-key-hex
```

For the existing Android Docker deployment, after updating the project files:

```
docker compose -f services/mobile-android/docker-compose.yml --project-directory . build api
frontend
docker compose -f services/mobile-android/docker-compose.yml --project-directory . up -d
docker compose -f services/mobile-android/docker-compose.yml --project-directory . exec api python
-m pytest -q tests/test_crypt14_pipeline_delivery.py tests/test_whatsapp_crypt.py
tests/test_whatsapp_crypt_v45.py tests/test_whatsapp_recovery.py tests/test_whatsapp_key_intake.py
```

These commands use the existing deployment configuration and prerequisites. Docker deployment and PowerShell execution were not run in this Linux validation environment.

Output files and exit codes

| Output | Purpose |

| --- | --- |

| decryption_report.json | Per-backup results, diagnostics, hashes, paths and counts; no raw keys |

| <source-path-hash>/msgstore.db | Successfully decoded SQLite database |

| <source-path-hash>/artifacts.jsonl | Normalized artifacts from the production WhatsApp parser |

| <source-path-hash>/matched_key.hex | Opt-in export of the already-collected authenticated matching AES key |

The source-path hash prevents backups with identical filenames from colliding. Exit code 0 means every discovered msgstore backup decoded; exit code 2 means at least one is blocked or none were found. Invalid input paths can exit with an exception. Do not reuse a previous successful output folder to assess a missing-key or tampering test: prior exports remain on disk. Use a fresh output folder for each test.

On Linux, exported files are created with owner-only permissions (0600). On Windows, apply the case folder's Windows ACL policy; chmod is not equivalent to a Windows owner-only ACL. The optional matching-key file is sensitive case material and must stay with controlled evidence.

Test cases and recorded results

| Test | Expected result | Validation |

| --- | --- | --- |

| Independent-format CRYPT14 + matching collected key | Valid database, expected hash, message artifact and optional key export | Passed |

| CRYPT14 without key | Blocked; no database or new key export | Passed |

| Wrong key | Blocked; no database or new key export | Passed |

| Modified authenticated bytes | Blocked; no database or new key export | Passed |

| Wrong candidate followed by matching candidate | Backup matches correct candidate | Passed |

| Parser preservation | Decoded bytes sent to storage, outcome recorded | Passed with storage mocked |

| Storage failure | Chats parsed, export marked failed | Passed with injected failure |

| Current-run status | Verified match becomes blocked when the new result fails; secrets omitted | Passed with persistence mocked |

| SQLite named .crypt14 | Readable records; no false authenticated key match | Passed |

103 relevant backend tests passed, with one dependency deprecation warning. Suites: test_crypt14_pipeline_delivery.py, test_whatsapp_crypt.py, test_whatsapp_crypt_v45.py, test_whatsapp_recovery.py, test_whatsapp_key_intake.py,

test_mobile_sqlite_counts.py, test_whatsapp_modern_extract.py and test_android_mobile_coverage.py. The narrower PowerShell regression launcher runs 88 tests.

Frontend TypeScript and Vite build passed (npm run build); existing bundle-size/dynamic-import warnings remain. Synthetic CLI demo passed: 1 discovered, 1 decoded, 0 blocked.

The broader existing test_disk_mobile_isolation.py has two failures in both the untouched input archive and the update: test_domain_stage_agents_registered expects disk-build but the registry uses disk-inventory; test_celery_routes_isolate_mobile_queue expects a literal supervisor route that the current factory no longer contains. Nine tests in that suite pass. Queue routing files were not modified.

Scope and limits

This does not promise every deleted row, every attachment, every database schema or every app's encrypted data. It decodes supported WhatsApp msgstore containers with matching key material and extracts supported records that are present. Media files must be acquired separately; other apps and device encryption have distinct keys and formats. CRYPT15 uses a different key derivation scheme; an exported raw CRYPT14 AES key should not be treated as a universal CRYPT15 password or key.

The pipeline currently reads an encrypted backup into memory with a 768 MB per-file read limit. The offline utility also uses memory for each whole backup and its decoded database. Larger backups need adequate memory and potentially a streaming implementation. Live-device acquisition, deployed PostgreSQL/MinIO integration and real customer backup validation were not executed. No real backup and matching key were supplied for this task.

Main changed files

backend/app/services/mobile_forensic/key_intake.py

backend/app/services/mobile_forensic/pipeline.py

backend/app/services/mobile_forensic/parsers/messaging.py

backend/app/services/mobile_forensic/whatsapp_case_decrypt.py

backend/scripts/test_crypt14_case.py

backend/tests/test_crypt14_pipeline_delivery.py

backend/tests/fixtures/crypt14_demo/

frontend/src/lib/types/forensic.ts

frontend/src/pages/forensic/JobIntakePage.tsx

Test-Crypt14.ps1 and Test-Crypt14.cmd